

9. SOLID 원칙 검토

9-1. SRP — 단일 책임 원칙

항목	내용
준수 여부	준수
적용 내용	<code>App.viewMySpecs()</code>는 단순히 사양 수집 시나리오의 흐름(대기 화면 출력 및 예외 처리 조율)만 처리하는 책임을 지며, 실제 클라이언트의 Windows/Linux Native 시스템 사양을 바이너리 수준에서 정밀하게 탐색하는 핵심 하드웨어 스캔 기능은 백엔드 인터페이스 레이어인 <code>window.electronAPI.collectSpecs()</code>가 단일 책임을 부여받아 수행하도록 하여 전은호 개발자가 안정적으로 구현하였습니다.
위반 내용 (해당 시)	초기 프로토타입 설계 시 App 객체가 직접 사용자 디바이스의 사양 텍스트 스트링값을 정규식으로 하나하나 추출하고 변형하는 일까지 한꺼번에 담당하여 SRP 위반 가능성이 있었음.
개선 방향 (해당 시)	자바스크립트의 DOM 렌더러 영역과 백엔드 데이터 가공 영역을 철저히 분리하여, 텍스트 가공 및 데이터 셋 빌딩은 수동 입력 처리부(<code>submitManual</code>) 및 시스템 브릿지 API 단에서 마친 뒤 App 은 정제된 객체 상태만 받아 렌더링하도록 분리해 해결함.

9-2. OCP — 개방-폐쇄 원칙

항목	내용
준수 여부	준수
적용 내용	하성권 분석가가 정의한 PC 사용 목적(게임, 업무, 학업 등)이 새롭게

	확장되는 상황이 닥치더라도, 핵심 비즈니스 로직 제어 루틴인 App.goToStep4나 App.runAnalysis 코드를 전혀 고치지 않고 외부 구성 정의 메타데이터 리터럴인 STEP4_CONFIG 구조체와 SUFFICIENCY 판단 임계 정적 데이터 맵에만 요소를 추가 작성해주는 방식으로 설계자 조우찬이 완전 격리 설계하여 준수했습니다.
위반 내용 (해당 시)	초기 구현 구상 시 목적이 늘어날 때마다 if-else 분기문을 추가하여 직접 조건문 안에서 질문 텍스트를 하나하나 수동 변환하여 매번 비즈니스 코드를 수정해야 하는 결합성이 발견됨.
개선 방향 (해당 시)	설정 데이터 기반 아키텍처(Configuration-Driven Design)로 변경하여, 질문 문항과 하위 옵션 배열을 정적 딕셔너리 데이터 구조인 STEP4_CONFIG 상수로 이관하고 렌더러는 이를 조회 및 대입만 하는 동적 파이프라인으로 개량함.

9-3. LSP — 리스코프 치환 원칙

항목	내용
준수 여부	준수
적용 내용	로컬 시스템 정보를 자동으로 감지하는 모드(App.selectThisPC())와 하드웨어 부품을 직접 입력하는 수동 품 모드(App.submitManual())가 완성하여 반환하는 최종 산출물인 state.specs 데이터 셋의 필드명과 규격을 완전 동일한 타입 포맷으로 조율했습니다. 이로 인해 최종 분석 요청(runAnalysis)을 수행하는 AI API 추상 인터페이스는 입력 출처에 상관없이 완벽하게 대체 작동됩니다.

위반 내용 (해당 시)	수동 입력창에서 사용자가 입력한 데이터는 일부 빈 필드나 문자열 조합 형태였고, 자동 감지 코드는 고도화된 객체 배열 형태여서 데이터 전송 시점의 시그니처 및 타입 일치성이 깨지는 모순이 일어났었음.
개선 방향 (해당 시)	App.submitManual 함수 하단부에 수동 파싱된 값들을 정형화된 모델 포맷으로 컨버팅하는 변환 구조 레이어를 삽입하여, 수동 입력을 거치더라도 자동 감지 프로세스와 완전히 치환 가능한 표준 규격의 specs 객체가 전역 상태(state.specs)에 주입되도록 보정함.